Assignment 4

Virtual Memory

20215174 임혁준

20225087 성준호

Submit : 2025.06.09

1. 문제 분석

PintOS 4번째 프로젝트의 목표는 가상 메모리(Virtual Memory)의 구현이다. 기존 PintOS에 구현되어 있는 Paging은 프로세스의 가상 주소에 free한 물리적 주소(프레임)를 대응시키고 부여하는 것에서 그친다. 즉, Virtual Memory 기능이 구현되지 않아, 물리 메모리가 가득 차게 되면 프로세스를 더 이상 실행시킬 수 없게 된다. 따라서 Demand Paging, Stack Growth, Swap과 같은 Virtual Memory 기능을 구현하는 것이 이번 프로젝트의 목표이고, 더 나아가 mmap, munmap 시스템 콜을 완성해야 한다.

2. 알고리즘 설계 및 구현

기존의 PintOS는 가상 주소를 Page의 Size만큼 분리해서 물리적 주소에 할당하는 기능(palloc)이 이미 구현된 상태이며, 이러한 Page들을 2-level page table(Page Directory -> Page Table -> Page)을 이용해 관리한다. 즉, Paging의 기초적인 기능은 이미 구현된 상태라고 볼 수 있다. 따라서 Virtual Memory를 위해 실질적으로 해야 하는 일은 기존의 구현된 Paging에 Demand Paging(Lazy Loading)과 Stack Growth, Swap(Frame Eviction)을 추가하는 것이라고 볼 수 있다. Pintos Stanford Manual에서는 이 기능들을 구현하기 위해 아래와 같은 3개의 Table을 추가할 것을 제안한다.

Supplemental page table
: Enables page fault handling by supplementing the page table. See Section 4.1.4 [Managing the Supplemental Page Table], page 42.

Frame table
: Allows efficient implementation of eviction policy. See Section 4.1.5 [Managing the Frame Table], page 42.

Swap table
: Tracks usage of swap slots. See Section 4.1.6 [Managing the Swap Table], page 43.

Fig 1. Needed Table Suggested by Pintos Stanford Manual

따라서 이번 프로젝트는 이 테이블들과 각각의 테이블의 기능을 구현하고 이들을 조합하여 Demand Paging(Lazy Loading)과 Stack Growth, Swap(Frame Eviction)을 구현하는 것이 주된 알고리즘 설계라고 볼 수 있다.

따라서 먼저 비어있는 src/vm 폴더에 각각의 테이블을 담당하는 코드를 추가해주었다.

```
# No virtual memory code yet.
vm_SRC  = vm/frame.c                    # Frame table.
vm_SRC += vm/page.c                     # Supplemental page table.
vm_SRC += vm/swap.c                     # Swap table.
```

Makefile.build

Supplemental Page Table (SPT)

Supplemental Page Table (SPT)은 Hash table로 구현하였다. 각각의 프로세스는 자신만의 SPT를 지닌다. SPT는 기존의 PintOS의 Page Table이 담고 있지 않은 Meta Data를 추가로 관리하여 Page fault, Swap과 같은 Page관련 작업에서 이러한 Meta Data를 통해 Paging을 보조하는 기능을 수행한다.

```c
enum page_status
{
    IN_FRAME,
    IN_SWAP,
    IN_DISK
};

struct page
{
    void* vaddr;                            /* Virtual address. */
    struct frame_entry* frame;          /* Occupying frame. */
    struct hash_elem hash_elem;     /* Hash table element. */

    /* Meta data. */
    enum page_status status;
    struct file* file;                      /* ELF excutable file to load page from */
    off_t offset;                           /* File offset */
    size_t read_bytes;              /* Number of bytes to read from file */
    bool writable;
    uint32_t* pagedir;

    /* Swap meta data. */
    int block_sector;
};
```

page.h

page.h파일에 Page 구조체를 정의한다.

vaddr	Page가 mapping 가상 주소(virtual address)
frame	Page가 mapping되어 있는 물리 메모리 프레임을 가리키는 포인터
hash_elem	SPT에 페이지를 등록하기 위한 elem 변수
status	Page의 현재 상태를 나타내기 위한 변수
file	Page가 속해있는 Process의 Segment 데이터를 담고 있는 파일 혹은 Page에 Mmap된 파일을 가리키는 파일.
offset	파일 내의 offset의 시작 위치. 즉, 파일 내의 어느 byte에서부터 write/read를 진행할지를 나타내는 변수.
read_bytes	Page가 파일로부터 실제로 읽은 바이트 수
writable	Page의 writable 여부를 나타내는 변수
pagedir	Page를 소유한 프로세스의 Page Directory 주소.
block_sector	Page가 Swap out될 때 Swap block의 섹터 번호를 저장하는 변수.

Thread 구조체에 SPT 를 선언하고, start_process() 함수 호출 시에 이를 hash_init()을 사용해 초기화한다.

```
#ifdef VM
    /* Supplemental page table made with hash table. */
    struct hash spt;
    uint32_t stack_pages;

    /* For mmap. */
    struct list mmap_list;
    int mapid_allocator;
#endif
```

thread.h

```
static void
start_process (void *file_name_)
{
  char *file_name = file_name_;
  struct intr_frame if_;
  bool success;

#ifdef VM
  hash_init(&thread_current()->spt, page_hash, page_less, NULL);
#endif
```

process.c

SPT에 저장된 Page의 관리를 위해, 해시값을 생성하는 함수 **page_hash()**와 Page간의 크기를 비교하는 함수 **page_less()**를 다음과 같이 정의한다.

```
/* hash function, address comparator */
/* Returns a hash value for page p. */
unsigned
page_hash(const struct hash_elem* p_, void* aux UNUSED)
{
    const struct page* p = hash_entry(p_, struct page, hash_elem);
    return hash_bytes(&p->vaddr, sizeof p->vaddr);
}

/* Returns true if page a precedes page b. */
bool
page_less(const struct hash_elem* a_, const struct hash_elem* b_,
    void* aux UNUSED)
{
    const struct page* a = hash_entry(a_, struct page, hash_elem);
    const struct page* b = hash_entry(b_, struct page, hash_elem);

    return pg_no(a->vaddr) < pg_no(b->vaddr);
}
```

page.c

```c
unsigned page_hash(const struct hash_elem*, void*);
bool page_less(const struct hash_elem*, const struct hash_elem*, void*);
void page_destructor(struct hash_elem*, void*);
struct page* page_find(struct hash*, void*);
```

page.h

Page가 삭제될 때 hash_destroy()에 사용되는 함수 page_destructor()와 SPT와 주소값이 주어질 때 Page를 탐색하는 함수 page_find()를 추가한다.

```c
void
page_destructor(struct hash_elem* e, void* aux UNUSED)
{
    struct page* p = hash_entry(e, struct page, hash_elem);
    if (p->frame != NULL) {
        struct frame_entry* f = p->frame;
        free_frame(f);
        p->frame = NULL;
    }
    if (p->block_sector != -1)
        swap_free(p);
    free(p);
}

/* Function to search the hash table. */
/* Returns the page containing the given virtual address, */
/* or a null pointer if no such page exists. */
struct page*
page_find(struct hash* spt, void* va)
{
    struct page page;
    struct hash_elem* e;

    page.vaddr = pg_round_down(va);

    e = hash_find(spt, &page.hash_elem);

    return e != NULL ? hash_entry(e, struct page, hash_elem) : NULL;
}
```

page.c

Frame Table

Frame Table은 Frame의 정보를 담고 있는 frame_entry 구조체를 만들어 malloc을 통해 kernel의 heap 영역에서 생성하고 관리한다. 더불어 bitmap인 free_frames를 통해 사용 가능한 Frame table의 특정 index에 대응하는 frame이 사용 가능한지 불가능한지를 판단한다. Frame Table에선 이들을 통해 Frame의 정보를 포함하고 있는 사용 가능한 물리 주소가 부족할 경우 Eviction을 진행하고 Swap을 진행하는 기능을 담당한다.

먼저 frame_entry 구조체의 정의를 완료하고, frame_init()을 선언해 frame을 초기화한다.

```c
struct frame_entry {
    uint32_t num;             // Frame number
    void* paddr;              // Frame physical address
    struct page* page_addr;      // Pointer to page that occupies the frame
};

void frame_init(size_t);
struct frame_entry* get_frame(void);
struct frame_entry* frame_get_multiple(size_t);
void free_frame(struct frame_entry*);
```

frame.h

```c
void
frame_init(size_t user_pages)
{
    size_t bm_pages = DIV_ROUND_UP(bitmap_buf_size(user_pages), PGSIZE);
    if (bm_pages > user_pages)
        bm_pages = user_pages;
    user_pages -= bm_pages;

    frame_table = (struct frame_entry*)malloc(sizeof(struct frame_entry) * user_pages);
    free_frames = bitmap_create(user_pages);
    size_t i;
    for (i = 0; i < user_pages; i++) {
        frame_table[i].num = i;
        frame_table[i].page_addr = NULL;
    }

    clock = 0;
    clock_max = (unsigned) user_pages;

    lock_init(&frame_lock);
}
```

frame.c

num	Frame의 Frame Table에서의 Index.
paddr	프레임에 대응되는 물리 주소.
Page_addr	Frame과 대응되는 Supplement Page Table Entry의 주소.

물리 프레임을 할당해서 반환하는 함수 frame_get_multiple(), 프레임을 반납하고 bitmap을 갱신하는 free_frame() 함수를 다음과 같이 정의한다.

```c
struct frame_entry*
frame_get_multiple(size_t page_cnt)
{
    lock_acquire(&frame_lock);
    size_t fnum = bitmap_scan_and_flip(free_frames, 0, page_cnt, false);
    if (fnum != BITMAP_ERROR) {
        frame_table[fnum].paddr = palloc_get_page(PAL_USER | PAL_ASSERT | PAL_ZERO);
        lock_release(&frame_lock);
        return &frame_table[fnum];
    }
    else {
        /* Frame eviction. */
        while (pagedir_is_accessed(frame_table[clock].page_addr->pagedir, frame_table[clock].page_addr->vaddr))
        {
            pagedir_set_accessed(frame_table[clock].page_addr->pagedir, frame_table[clock].page_addr->vaddr, false)
            clock = (clock + 1) % clock_max;
        }

        /* Swap frame. */
        struct page* victim_page = frame_table[clock].page_addr;

        swap_insert(victim_page);
        victim_page->frame = NULL;
        victim_page->status = IN_SWAP;
        pagedir_clear_page(victim_page->pagedir, victim_page->vaddr);

        unsigned clock_prev = clock;
        clock = (clock + 1) % clock_max;
        lock_release(&frame_lock);
        return &frame_table[clock_prev];
    }
}
```

```c
struct frame_entry*
get_frame()
{
    return frame_get_multiple(1);
}

void
free_frame(struct frame_entry* f)
{
    lock_acquire(&frame_lock);
    pagedir_clear_page(f->page_addr->pagedir, f->page_addr->vaddr);
    bitmap_reset(free_frames, f->num);
    palloc_free_page(frame_table[f->num].paddr);
    f->page_addr = NULL;
    lock_release(&frame_lock);
}
```

frame.c

Swap Table

Swap table은 frame에서 eviction 및 swap이 진행될 경우 해당 frame에 대응되는 Page를 디스크의 Swap block에 저장하고 메모리로 가져오거나 디스크에서 메모리로 다시 가져오는 기능을 수행한다. Frame table에서와 마찬가지로 사용 가능한 Swap block을 나타내기 위해 bitmap인 free_blocks를 정의하였다. Swap Table의 기능을 구현할 때 devices/block.c의 기능들을 사용하여 Disk와 Memory간의 데이터 작업을 수행하도록 하였다.

```
--swap-size=4
```

PintOS에서는 test를 진행할 때 swap-size, 즉, Swap disk로 사용할 Disk의 크기를 4MB로 할당하는 것을 알 수 있다. 4MB Swap Disk는 Page Size인 4KB의 1024 (4MB/4KB) 개의 Swap Block으로 구성되고, Block은 512 Bytes의 8 (4KB/512B) 개의 Sector로 구성되며, PintOS에서는 Block에 write/read를 진행할 때 Sector 단위로 진행한다. 따라서 이를 전제로 Swap 기능을 구현해야 한다.

```
void
swap_init(void)
{
    free_blocks = bitmap_create(1024);
    swap_table = block_get_role(BLOCK_SWAP);
    lock_init(&block_lock);
}
```

swap.c

swap_init() 함수를 통해 swap 시스템을 초기화하고 swap에 사용될 공간을 지정한다. 이 함수는 thread/init.c의 main()에서 호출된다. Bitmap인 free_blocks의 경우 1024로 할당하는데, Swap Block이 1024개 존재하기 때문이다.

```
#ifdef VM
  swap_init();
#endif
```

init.c

물리 프레임을 swap 영역에 삽입하는 함수 swap_insert()을 작성한다. bitmap_scan_and_flip으로 free swap 공간을 찾고, block_write()를 통해 Page를 swap 영역에 기록한다. write/read는 Sector 단위로 진행한다.

```c
/* Insert the contents of a page into the swap table */
void
swap_insert(struct page* p)
{
    lock_acquire(&block_lock);
    char* c = (char*) p->frame->paddr;
    size_t sector_num = bitmap_scan_and_flip(free_blocks, 0, 1, false);
    if (sector_num == BITMAP_ERROR)
        PANIC("Swap space is full.");
    p->block_sector = sector_num;
    int i;
    for (i = 0; i < 8; i++) {
        block_write(swap_table, sector_num * 8 + i, c);
        c += 512;
    }
    lock_release(&block_lock);
}
```

swap.c

swap영역에 저장된 페이지를 메모리로 다시 불러오는 함수 swap_get()을 작성한다. block_read()로 저장된 Page를 로딩한 뒤, bitmap_reset()을 통해 해당 슬롯을 free처리한다.

```c
void
swap_get(struct page* p)
{
    lock_acquire(&block_lock);
    char* c = (char*) p->frame->paddr;
    size_t read_sector = p->block_sector;
    int i;
    for (i = 0; i < 8; i++) {
        block_read(swap_table, read_sector * 8 + i, c);
        c += 512;
    }
    bitmap_reset(free_blocks, read_sector);
    p->block_sector = -1;
    lock_release(&block_lock);
}
```

swap.c

```c
void
swap_free(struct page* p)
{
    lock_acquire(&block_lock);
    bitmap_reset(free_blocks, p->block_sector);
    lock_release(&block_lock);
}
```

swap.c

Swap Block을 해제하는 함수 swap_free()를 작성한다. Page가 Swap Disk에 존재하는 상태에서 해제되는 경우에서 사용된다.

Problem 1-2. Demand Paging (Lazy Loading), Stach Growth, Swap 구현

Demand Paging (Lazy Loading)

기존의 PintOS는 프로세스의 Code, Data segment가 Process가 load될 때 바로 Frame에 load되는 형태로, 프로세스가 특정 segment를 필요로 하지 않음에도 Frame에 올려두는 Demand Paging이 구현되지 않은 상태였다. 따라서 프로세스가 load될 때 먼저 Page에 segment의 내용을 저장만 해두고, 실질적인 frame 할당은 프로세스가 해당 segment를 필요로 하여 page fault가 발생했을 때 진행시키는 Demand Paging을 구현해야 한다. 아래의 코드의 초록 상자 부분이 Demand Paging코드이고, 빨간 상자가 기존의 코드이다.

```c
static bool
load_segment (struct file *file, off_t ofs, uint8_t *upage,
              uint32_t read_bytes, uint32_t zero_bytes, bool writable)
{
  ASSERT ((read_bytes + zero_bytes) % PGSIZE == 0);
  ASSERT (pg_ofs (upage) == 0);
  ASSERT (ofs % PGSIZE == 0);

  while (read_bytes > 0 || zero_bytes > 0)
    {
      /* Calculate how to fill this page.
         We will read PAGE_READ_BYTES bytes from FILE
         and zero the final PAGE_ZERO_BYTES bytes. */
      size_t page_read_bytes = read_bytes < PGSIZE ? read_bytes : PGSIZE;
      size_t page_zero_bytes = PGSIZE - page_read_bytes;

      #ifdef VM
        struct page* p = (struct page*)malloc(sizeof(struct page));
        p->vaddr = upage;
        p->frame = NULL;
        p->status = IN_DISK;
        p->file = file;
        p->offset = ofs;
        p->read_bytes = page_read_bytes;
        p->writable = writable;
        p->pagedir = thread_current()->pagedir;
        p->block_sector = -1;
        hash_insert(&thread_current()->spt, &p->hash_elem);
      #else
        /* Get a page of memory. */
        uint8_t *kpage = palloc_get_page (PAL_USER);

        if (kpage == NULL)
          return false;

        /* Load this page. */
        if (file_read (file, kpage, page_read_bytes) != (int) page_read_bytes)
          {
            palloc_free_page(kpage);
            return false;
          }
        memset (kpage + page_read_bytes, 0, page_zero_bytes);

        /* Add the page to the process's address space. */
        if (!install_page(upage, kpage, writable))
        {
            palloc_free_page(kpage);
            return false;
        }
      #endif
      /* Advance. */
      read_bytes -= page_read_bytes;
      zero_bytes -= page_zero_bytes;
      ofs += page_read_bytes;
      upage += PGSIZE;
    }
  return true;
}
```

userprog/procces.c

Page fault 발생 시, fault address (virtual address)에 해당하는 Page가 IN_DISK 상태일 때, Frame을 할당하고 Loading을 진행한다.

```
static void
page_fault (struct intr_frame *f)
{
  bool not_present;  /* True: not-present page, false: writing r/o page. */
  bool write;        /* True: access was write, false: access was read. */
  bool user;         /* True: access by user, false: access by kernel. */
  void *fault_addr;  /* Fault address. */

  /* Obtain faulting address, the virtual address that was
     accessed to cause the fault.  It may point to code or to
     data.  It is not necessarily the address of the instruction
     that caused the fault (that's f->eip).
     See [IA32-v2a] "MOV--Move to/from Control Registers" and
     [IA32-v3a] 5.15 "Interrupt 14--Page Fault Exception
     (#PF)". */
  asm ("movl %%cr2, %0" : "=r" (fault_addr));

  /* Turn interrupts back on (they were only off so that we could
     be assured of reading CR2 before it changed). */
  intr_enable ();

  /* Count page faults. */
  page_fault_cnt++;

  /* Determine cause. */
  not_present = (f->error_code & PF_P) == 0;
  write = (f->error_code & PF_W) != 0;
  user = (f->error_code & PF_U) != 0;

  ////////////////////////////////////////////////////////////// ADDED

#ifdef VM
  struct thread* t = thread_current();
  bool held_filesys_lock = lock_held_by_current_thread(&filesys_lock);

  if (fault_addr == NULL || is_kernel_vaddr(fault_addr))
     thread_exit();

  struct page* p = page_find(&t->spt, fault_addr);
```

```
if (p->status == IN_DISK)
{
    struct frame_entry* frame = get_frame();
    uint8_t* kpage = frame->paddr;
    frame->page_addr = p;

    if (!held_filesys_lock)
      lock_acquire(&filesys_lock);
    if (file_read_at(p->file, kpage, p->read_bytes, p->offset) != (int)p->read_bytes)
    {
        lock_release(&filesys_lock);
        thread_exit();
    }
    if (!held_filesys_lock)
      lock_release(&filesys_lock);

    size_t zero_bytes = PGSIZE - p->read_bytes;
    memset(kpage + p->read_bytes, 0, zero_bytes);

    if (!(pagedir_get_page(t->pagedir, p->vaddr) == NULL
        && pagedir_set_page(t->pagedir, p->vaddr, kpage, p->writable)))
    {
        free_frame(f);
        hash_delete(&t->spt, &p->hash_elem);
        free(p);
        thread_exit();
    }

    p->status = IN_FRAME;
    p->frame = frame;

    return;
}
```

userprog/exception.c

Stack Growth

Page fault 시, fault address에 Page가 없을 때, 새 Page를 동적으로 생성해 stack을 복구한다. 이때, Stack Growth의 제한 값 이내인지 확인해야 한다.

```
#ifdef VM
  struct thread* t = thread_current();
  bool held_filesys_lock = lock_held_by_current_thread(&filesys_lock);

  if (fault_addr == NULL || is_kernel_vaddr(fault_addr))
      thread_exit();

  struct page* p = page_find(&t->spt, fault_addr);

   if (p == NULL)
   {
       /* If fault_addr is outside of stack range (invalid), exit. */
       char* esp = f->esp;
       if ((fault_addr > PHYS_BASE) || (fault_addr < PHYS_BASE - 0x800000) ||
           (fault_addr > (void*)(esp + 32)) || (fault_addr < (void*)(esp - 32)))
           thread_exit();

       /* Grow stack page. */
       char* current_stack = PHYS_BASE - (t->stack_pages * PGSIZE);
       char* new_page_addr = (char*) pg_round_down(fault_addr);
       for (; new_page_addr < current_stack; new_page_addr += PGSIZE) {
           struct page* nsp = (struct page*)malloc(sizeof(struct page));
           nsp->vaddr = new_page_addr;
           nsp->status = IN_FRAME;
           nsp->file = NULL;
           nsp->offset = 0;
           nsp->read_bytes = 0;
           nsp->writable = true;
           nsp->pagedir = t->pagedir;
           nsp->block_sector = -1;
           hash_insert(&t->spt, &nsp->hash_elem);

           t->stack_pages++;
           if (t->stack_pages > 2048)
           {
               hash_delete(&t->spt, &nsp->hash_elem);
               free(nsp);
               thread_exit();
           }

           struct frame_entry* stack_frame = get_frame();
           stack_frame->page_addr = nsp;

```

```
            /* Install. */
            if (!(pagedir_get_page(t->pagedir, nsp->vaddr) == NULL
                && pagedir_set_page(t->pagedir, nsp->vaddr, stack_frame->paddr, nsp->writable)))
            {
                free_frame(f);
                hash_delete(&t->spt, &nsp->hash_elem);
                free(nsp);
                thread_exit();
            }

            nsp->frame = stack_frame;
        }

        return;
    }
```

exception.c

주소값을 먼저 확인하여, 잘못된 주소로의 접근의 경우 Page 탐색을 실시하지 않고 exit()한다. 새로이 생성된 Page는 IN_FRAME의 status로 생성되고, frame 할당까지 바로 진행된다. 만약 할당 과정에서 실패한 경우, 할당을 취소하고 exit()한다.

Swap

Page fault 발생 시 Page가 Swap Disk에 있었다면, 다시 메모리로 load한다. 이 과정에서 프레임을 새로 할당해준다.

```
if (p->status == IN_SWAP)
{
    struct frame_entry* frame = get_frame();
    uint8_t* kpage = frame->paddr;
    frame->page_addr = p;
    p->frame = frame;

    swap_get(p);

    if (!(pagedir_get_page(t->pagedir, p->vaddr) == NULL
        && pagedir_set_page(t->pagedir, p->vaddr, kpage, p->writable)))
    {
        free_frame(f);
        hash_delete(&t->spt, &p->hash_elem);
        free(p);
        thread_exit();
    }

    p->status = IN_FRAME;

    return;
}
```

userprog/exception.c

Problem 3. Memory Mapped Files

과제 4 에서는 VM 과 더불어 mmap 과 munmap 의 구현을 요구한다. 이를 위해서 Pintos Stanford Manual 에서 제안하는 mmap table 을 구현하고, mmap 과 munmap 기능 및 system call 을 구현한다.

Table of file mappings

Processes may map files into their virtual memory space. You need a table to track which files are mapped into which pages.

Fig 2. Needed Table for MMAP Suggested by Pintos Stanford Manual

Mmap table 을 위한 mmap.c 코드를 src/vm 폴더에 추가한다.

```
vm_SRC += vm/mmap.c                    # MMAP table.
```

syscall_handler 에 SYS_MMAP 과 SYS_MUNMAP 을 처리하기 위한 case 구문을 추가한다.

```
#ifdef VM
    case SYS_MMAP:
        check_user_vaddr(sp + 4);
        f->eax = mmap((int)*(uint32_t*)(sp + 4), (void*)*(uint32_t*)(sp + 8));
        break;

    case SYS_MUNMAP:
        check_user_vaddr(sp + 4);
        munmap((int)*(uint32_t*)(sp + 4));
        break;
#endif
```

```
#ifdef VM

mapid_t
mmap(int fd, void* addr)
{
    if (addr == NULL || pg_ofs(addr) != 0 || !is_user_vaddr(addr))
        return -1;
    mapid_t mapid = do_mmap(fd, addr);
    return mapid;
}

void
munmap(mapid_t mapid)
{
    if (mapid < 0)
        return -1;
    do_munmap(mapid);
}

#endif
```

syscall.c

mmap 을 실시하기 전, 유효한 가상주소인지, Page 정렬된 주소인지, 사용자 영역 주소인지 확인한다. 그렇지 않다면 return -1; 을 해준다.

메모리 매핑에 대한 정보를 저장하는 구조체 struct mmap_entry 를 선언한다. Mmap Table 은 list 형태로 process 마다 개별 할당하여 관리하도록 한다.

```
struct mmap_entry {
    int mapid;
    struct page** page_addrs;
    int pg_cnt;
    struct file* file;
    struct list_elem elem;
};
```

mmap.h

mmap.c 에서, do_mmap 작성을 완료한다. 파일을 읽을 수 없었거나, 파일의 길이가 0 이라면 즉시 return 한다. 읽을 수 있었다면, 파일이 존재하는 영역의 페이지가 이미 할당되어 SPT 에 존재하는 지 찾는다. 만약 Page 가 이미 할당되어 있다면 즉시 반환한다.

```
int
do_mmap(int fd, void* addr)
{
    struct file* origin = getfile(fd);
    if (origin == NULL)
    {
        return -1;
    }

    struct file* f = file_reopen(origin);
    if (f == NULL)
    {
        return -1;
    }

    off_t ofs = f->pos;
    off_t length = file_length(f);
    if (length == 0)
    {
        return -1;
    }

    struct thread* t = thread_current();

    size_t read_bytes = length;
    size_t zero_bytes = (PGSIZE - read_bytes % PGSIZE) % PGSIZE;

    int pg_cnt = length <= PGSIZE ? 1 : length % PGSIZE ? length / PGSIZE + 1 : length / PGSIZE;

    int i;
    for (i = 0; i < pg_cnt; i++) {
        void* check_addr = addr + i * PGSIZE;
        if (page_find(&t->spt, check_addr) != NULL)
            return -1;
    }
```

mmap.c

이후 파일의 각 영역에 대한 Page 를 생성해 SPT 에 저장한다. 그리고 메모리 매핑 정보를 mmap_list 에 저장한다.

```c
        struct mmap_entry* me = (struct mmap_entry*)malloc(sizeof(struct mmap_entry));
        int mapid = t->mapid_allocator++;
        me->mapid = mapid;
        me->pg_cnt = pg_cnt;
        me->file = f;
        me->page_addrs = (struct page**)malloc(sizeof(struct page*) * pg_cnt);
        list_push_back(&t->mmap_list, &me->elem);

        int j;
        while (read_bytes > 0 || zero_bytes > 0)
        {
            /* Calculate how to fill this page.
               We will read PAGE_READ_BYTES bytes from FILE
               and zero the final PAGE_ZERO_BYTES bytes. */
            size_t page_read_bytes = read_bytes < PGSIZE ? read_bytes : PGSIZE;
            size_t page_zero_bytes = PGSIZE - page_read_bytes;

            struct page* p = (struct page*)malloc(sizeof(struct page));
            p->vaddr = addr;
            p->frame = NULL;
            p->status = IN_DISK;
            p->file = f;
            p->offset = ofs;
            p->read_bytes = page_read_bytes;
            p->writable = true;
            p->pagedir = thread_current()->pagedir;
            p->block_sector = -1;

            hash_insert(&t->spt, &p->hash_elem);
            me->page_addrs[j++] = p;

            read_bytes -= page_read_bytes;
            zero_bytes -= page_zero_bytes;
            ofs += page_read_bytes;
            addr += PGSIZE;
        }

        return mapid;
    }
```

mmap.c

do_munmap 함수 작성을 완료한다.

현재 실행중인 process 로부터 mmap_list 를 얻어와 메모리 매핑에 대한 정보를 얻어온다. mmap_list 에서 id 가 일치하는 mmap_entry 구조체를 찾으면, 해당 메모리 매핑을 참고해 메모리를 해제한다. 만약 해당 Page 가 dirty 라면, 해당 구역에 file_write_at()을 실행해 파일을 작성한다. free_frame()함수를 호출해 frame 을 해제하고 SPT 테이블에서 해당 Page 를 제거해 메모리를 최적화해야 한다.

```
void
do_munmap(int mapid)
{
    struct thread* t = thread_current();

    struct list_elem* e;
    for (e = list_begin(&t->mmap_list); e != list_end(&t->mmap_list);)
    {
        struct mmap_entry* me = list_entry(e, struct mmap_entry, elem);
        e = list_next(e);

        if (mapid != -1 && me->mapid != mapid)
            continue;

        int i;
        for (i = 0; i < me->pg_cnt; i++)
        {
            struct page* p = me->page_addrs[i];
            if (p->frame != NULL)
            {
                if (pagedir_is_dirty(p->pagedir, p->vaddr))
                {
                    if (file_write_at(p->file, p->frame->paddr, p->read_bytes, p->offset) != (int)p->read_bytes)
                        thread_exit();
                }
                if (mapid != -1)
                    free_frame(p->frame);
            }
            if (mapid != -1)
            {
                hash_delete(&t->spt, &p->hash_elem);
                free(p);
            }
        }
        list_remove(&me->elem);
        free(me->page_addrs);
        free(me);
    }
}
```

mmap.c

do_munmap()의 인자로 mapid=-1 이 들어올 경우, 메모리 매핑된 모든 페이지에 대해 매핑을 삭제한다. 이는 process_exit()시 모든 메모리 매핑을 해제해야 하기 때문이다.

```
#ifdef VM
  do_munmap(-1);
  hash_destroy(&cur->spt, page_destructor);
#endif
```

process.c

Problem 4. Accessing User Memory

System call에서 read()와 같은 함수에서 유저 메모리 영역에 접근을 시도하거나, lock과 같은 자원을 지니고 유저 메모리 영역에 접근할 때, 해당 Page에 대해 Page Fault가 발생하는 것을 막거나, 따로 처리해야 한다. 또한 해당 페이지에 할당된 프레임이 eviction 되는 것을 막아야 한다. 이를 Pinning 또는 Locking으로 구현해야 한다.

하지만 현재 구현한 코드에서는 이를 따로 구현하지는 않고 단순하게 lock의 여부를 판단하고 lock 재습득 판단에 대한 코드만 추가하는 정도로 Dead Lock state만 방지하였다. Test case에서는 모두 통과하였지만, 추후 Pinning을 사용해서 구현해야 할 기능임이 분명하다.

```
if (!held_filesys_lock)
  lock_acquire(&filesys_lock);
if (file_read_at(p->file, kpage, p->read_bytes, p->offset) != (int)p->read_bytes)
{
    lock_release(&filesys_lock);
    thread_exit();
}
if (!held_filesys_lock)
  lock_release(&filesys_lock);
```

userprog/exception.c

3. 결과

```
make[1]: Entering directory `/home/junho/pintos/src/vm/build'
pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
pass tests/userprog/sc-bad-sp
pass tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
pass tests/userprog/halt
pass tests/userprog/exit
pass tests/userprog/create-normal
pass tests/userprog/create-empty
pass tests/userprog/create-null
pass tests/userprog/create-bad-ptr
pass tests/userprog/create-long
```

```
pass tests/userprog/create-exists
pass tests/userprog/create-bound
pass tests/userprog/open-normal
pass tests/userprog/open-missing
pass tests/userprog/open-boundary
pass tests/userprog/open-empty
pass tests/userprog/open-null
pass tests/userprog/open-bad-ptr
pass tests/userprog/open-twice
pass tests/userprog/close-normal
pass tests/userprog/close-twice
pass tests/userprog/close-stdin
pass tests/userprog/close-stdout
pass tests/userprog/close-bad-fd
pass tests/userprog/read-normal
pass tests/userprog/read-bad-ptr
pass tests/userprog/read-boundary
pass tests/userprog/read-zero
pass tests/userprog/read-stdout
pass tests/userprog/read-bad-fd
pass tests/userprog/write-normal
pass tests/userprog/write-bad-ptr
pass tests/userprog/write-boundary
pass tests/userprog/write-zero
```

```
pass tests/userprog/write-stdin
pass tests/userprog/write-bad-fd
pass tests/userprog/exec-once
pass tests/userprog/exec-arg
pass tests/userprog/exec-multiple
pass tests/userprog/exec-missing
pass tests/userprog/exec-bad-ptr
pass tests/userprog/wait-simple
pass tests/userprog/wait-twice
pass tests/userprog/wait-killed
pass tests/userprog/wait-bad-pid
pass tests/userprog/multi-recurse
pass tests/userprog/multi-child-fd
pass tests/userprog/rox-simple
pass tests/userprog/rox-child
pass tests/userprog/rox-multichild
pass tests/userprog/bad-read
pass tests/userprog/bad-write
pass tests/userprog/bad-read2
pass tests/userprog/bad-write2
pass tests/userprog/bad-jump
pass tests/userprog/bad-jump2
pass tests/vm/pt-grow-stack
pass tests/vm/pt-grow-pusha
```

```
pass tests/vm/pt-grow-bad
pass tests/vm/pt-big-stk-obj
pass tests/vm/pt-bad-addr
pass tests/vm/pt-bad-read
pass tests/vm/pt-write-code
pass tests/vm/pt-write-code2
pass tests/vm/pt-grow-stk-sc
pass tests/vm/page-linear
pass tests/vm/page-parallel
pass tests/vm/page-merge-seq
pass tests/vm/page-merge-par
pass tests/vm/page-merge-stk
pass tests/vm/page-merge-mm
pass tests/vm/page-shuffle
pass tests/vm/mmap-read
pass tests/vm/mmap-close
pass tests/vm/mmap-unmap
pass tests/vm/mmap-overlap
pass tests/vm/mmap-twice
pass tests/vm/mmap-write
pass tests/vm/mmap-exit
pass tests/vm/mmap-shuffle
pass tests/vm/mmap-bad-fd
pass tests/vm/mmap-clean
```

```
pass tests/vm/mmap-inherit
pass tests/vm/mmap-misalign
pass tests/vm/mmap-null
pass tests/vm/mmap-over-code
pass tests/vm/mmap-over-data
pass tests/vm/mmap-over-stk
pass tests/vm/mmap-remove
pass tests/vm/mmap-zero
pass tests/filesys/base/lg-create
pass tests/filesys/base/lg-full
pass tests/filesys/base/lg-random
pass tests/filesys/base/lg-seq-block
pass tests/filesys/base/lg-seq-random
pass tests/filesys/base/sm-create
pass tests/filesys/base/sm-full
pass tests/filesys/base/sm-random
pass tests/filesys/base/sm-seq-block
pass tests/filesys/base/sm-seq-random
pass tests/filesys/base/syn-read
pass tests/filesys/base/syn-remove
pass tests/filesys/base/syn-write
All 109 tests passed.
make[1]: Leaving directory `/home/junho/pintos/src/vm/build'
junho@ubuntu:~/pintos/src/vm$
```

실행 결과, 109개의 테스트 케이스를 모두 성공적으로 통과할 수 있었다.